Chapter 3 — Python for AI Engineers

3.1 Asyncio and Concurrency for I/O-Bound LLM Calls

Conceptual Deep Dive

LLM calls are I/O-bound: the CPU waits on the network. `asyncio` runs one thread with an event loop; `await` yields control so thousands of requests overlap. Key primitives: coroutines, `asyncio.gather`, `asyncio.TaskGroup` (3.11+, structured concurrency with cancellation on failure), `asyncio.Semaphore` (bound concurrency to respect rate limits), `asyncio.timeout`, and queues for producer/consumer pipelines. Blocking calls (`requests`, `time.sleep`, heavy CPU) freeze the loop; push them to `asyncio.to_thread` or a process pool.

Threads vs processes vs async: async for network I/O, threads for blocking libraries, processes (`multiprocessing`, `ProcessPoolExecutor`) for CPU-bound work because of the **GIL**. (Python 3.13 offers an experimental free-threaded build.)

Cross-Connection Scenarios

Chapter 4 layers retries and rate limits on this; Chapter 10 uses it to run evals in parallel; Chapter 9 uses it for batch embedding.

Production Anti-Patterns

- Unbounded `gather` over 10,000 tasks → 429s and memory blowup.
- Calling blocking SDKs inside `async def`.
- Swallowing `CancelledError`.
- Creating a new HTTP client per request (no connection reuse).

Minimal Working Code

```
import asyncio
import httpx

async def fetch(client: httpx.AsyncClient, sem: asyncio.Semaphore, url: str) -> int:
    async with sem:
        r = await client.get(url, timeout=10)
        return r.status_code

async def run(urls: list[str], limit: int = 20) -> list[int | BaseException]:
    sem = asyncio.Semaphore(limit)
    async with httpx.AsyncClient() as client:
        return await asyncio.gather(*(fetch(client, sem, u) for u in urls),
                                    return_exceptions=True)

if __name__ == "__main__":
    print(asyncio.run(run(["https://example.com"] * 3)))
```

3.2 Generators, Iterators and Streaming

Conceptual Deep Dive

A generator produces values lazily, holding O(1) memory. Async generators (`async def` + `yield`) are the natural shape of token streams. Use them to stream LLM output, read huge JSONL datasets, and build pipelines (read → clean → chunk → embed) without loading everything. itertools (islice, batched in 3.12, chain) composes them.

Cross-Connection Scenarios

SSE streaming (4), chunking pipelines (9), dataset iteration for fine-tuning (12).

Production Anti-Patterns

- `list(generator)` on a 50 GB file.
- Forgetting that generators are single-use.
- Not closing async generators on client disconnect (resource leak).

Minimal Working Code

```
import json
from itertools import islice
from typing import Iterable, Iterator

def read_jsonl(path: str) -> Iterator[dict]:
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield json.loads(line)

def batched(it: Iterable, n: int) -> Iterator[list]:
    it = iter(it)
    while batch := list(islice(it, n)):
        yield batch

async def fake_token_stream(text: str):
    import asyncio
    for tok in text.split():
        await asyncio.sleep(0)
        yield tok + " "
```

3.3 Type Hints, Dataclasses and Pydantic

Conceptual Deep Dive

Type hints document and let tools (mypy, pyright) catch bugs before runtime. Protocol gives structural typing (swap LLM clients without inheritance), TypedDict types JSON-like dicts, Literal and Enum constrain values, generics (TypeVar) type containers. dataclass is for internal data; **pydantic v2** validates untrusted data (LLM output, API payloads) and generates JSON Schema — the bridge to Chapter 7.

Cross-Connection Scenarios

Pydantic models double as tool schemas (7), config (15), and API request bodies (4).

Production Anti-Patterns

- `Any` everywhere.
- Trusting LLM JSON without validation.
- Mutable default arguments.

Minimal Working Code

```
from typing import Protocol
from pydantic import BaseModel, Field

class LLM(Protocol):
    def complete(self, prompt: str) -> str: ...

class Ticket(BaseModel):
    category: str = Field(pattern="^(billing|bug|other)$")
    urgency: int = Field(ge=1, le=5)

def classify(llm: LLM, text: str) -> Ticket:
    raw = llm.complete(f"Return JSON with category,urgency for: {text}")
    return Ticket.model_validate_json(raw)

class Stub:
    def complete(self, prompt: str) -> str:
        return '{"category":"bug","urgency":4}'

print(classify(Stub(), "app crashes"))
```

3.4 Memory Management and Profiling

Conceptual Deep Dive

CPython uses reference counting plus a cyclic garbage collector. Memory problems in AI code: loading full datasets, accumulating embeddings in lists of Python floats (use NumPy float32 arrays: 4 bytes vs ~32+), holding references in caches that never evict, and GPU tensors kept alive by lingering references. Tools: tracemalloc, memory_profiler, sys.getsizeof, __slots__, functools.lru_cache(maxsize=...), memory-mapped arrays (np.memmap), and weakref.

Cross-Connection Scenarios

Vector storage sizing (8), model memory (5, 13), dataset streaming (12).

Production Anti-Patterns

- Unbounded in-process caches / conversation histories.
- float64 embeddings by accident.
- Not profiling before "optimising".

Minimal Working Code

```
import tracemalloc
import numpy as np

tracemalloc.start()
vecs = np.random.rand(100_000, 384).astype(np.float32)
cur, peak = tracemalloc.get_traced_memory()
print(f"{vecs.nbytes/1e6:.1f} MB array; traced peak {peak/1e6:.1f} MB")

class Msg:
    __slots__ = ("role", "content")
    def __init__(self, role: str, content: str):
        self.role, self.content = role, content
```

3.5 High-Performance Data Manipulation

Conceptual Deep Dive

Vectorise: replace Python loops with NumPy/pandas/Polars operations that run in C/Rust. Broadcasting, einsum, and matrix multiplication give similarity search over millions of vectors. Polars (lazy, multi-threaded, Arrow-backed) and DuckDB handle datasets larger than pandas comfortably. Parquet/Arrow beat CSV for size and speed. For CPU-bound pure Python use ProcessPoolExecutor; for hot loops consider Numba.

Cross-Connection Scenarios

Embedding similarity (8), dataset prep for SFT (12), eval result aggregation (10).

Production Anti-Patterns

- df.apply(lambda ...) row-by-row on millions of rows.
- Storing datasets as giant CSVs.
- Copying arrays unnecessarily (.copy() in loops).

Minimal Working Code

```
import numpy as np

def top_k_cosine(query: np.ndarray, matrix: np.ndarray, k: int = 5):
    q = query / np.linalg.norm(query)
    m = matrix / np.linalg.norm(matrix, axis=1, keepdims=True)
    scores = m @ q
    idx = np.argpartition(-scores, k)[:k]
    return idx[np.argsort(-scores[idx])], scores[idx[np.argsort(-scores[idx])]]

rng = np.random.default_rng(0)
M = rng.standard_normal((10_000, 64)).astype(np.float32)
print(top_k_cosine(M[0], M, 3))
```

3.6 Multiprocessing, Packaging and Environment Hygiene

Conceptual Deep Dive

multiprocessing sidesteps the GIL by using separate processes; data is pickled across boundaries, so pass small things. Use spawn start method for CUDA safety. Packaging: pyproject.toml, uv or pip-tools for lockfiles, virtual environments per project, pinned versions for reproducibility. Logging via logging (structured JSON), config via env vars (12-factor), secrets never in code.

Cross-Connection Scenarios

Reproducibility (15), containers (13), observability (14).

Production Anti-Patterns

- Unpinned dependencies; "works on my machine".
- Forking after CUDA initialisation.
- print debugging in services.

Minimal Working Code

```
from concurrent.futures import ProcessPoolExecutor
import logging
import os

logging.basicConfig(level=os.getenv("LOG_LEVEL", "INFO"),
                    format='{"t":"%(asctime)s","lvl":"%(levelname)s&qu
ot;,"msg":"%(message)s"}')
log = logging.getLogger("pipeline")
```

```
def cpu_heavy(n: int) -> int:
    return sum(i * i for i in range(n))

if __name__ == "__main__":
    with ProcessPoolExecutor() as ex:
        print(list(ex.map(cpu_heavy, [10**5, 10**6])))
    log.info("done")
```

3.7 Advanced Async Patterns: Queues, Backpressure, Cancellation

Conceptual Deep Dive

A bounded asyncio.Queue gives **backpressure**: producers block when consumers are slow, so memory stays flat. Worker-pool pattern: N consumers pull from the queue, and queue.join() signals completion. **TaskGroup** (3.11+) cancels siblings when one fails; asyncio.timeout() bounds a block; always re-raise CancelledError after cleanup. asyncio.as_completed yields results as they finish, useful for streaming partial eval results. Edge cases: exceptions in gather without return_exceptions leave other tasks running; create_task without keeping a reference lets the task be garbage-collected; blocking sqlite3/file I/O stalls the loop; contextvars carry request ids across tasks.

Cross-Connection Scenarios

Rate-limited batch embedding (8.6), parallel eval runs (10.6), tool parallelism (7.5).

Production Anti-Patterns

- Unbounded queues that absorb a whole dataset in RAM.
- Fire-and-forget tasks whose exceptions vanish.
- Catching Exception and swallowing cancellation.

Minimal Working Code

```
import asyncio

async def worker(name, q: asyncio.Queue, results: list):
    while True:
        item = await q.get()
        try:
            await asyncio.sleep(0.01)          # stands in for an API call
            results.append((name, item))
        finally:
            q.task_done()

async def run(items, n_workers=4, maxsize=8):
    q, results = asyncio.Queue(maxsize=maxsize), []
    workers = [asyncio.create_task(worker(i, q, results)) for i in range(n_workers)]
    for it in items:
        await q.put(it)                        # blocks when full -> backpressure
    await q.join()
    for w in workers:
        w.cancel()
    await asyncio.gather(*workers, return_exceptions=True)
    return results

print(len(asyncio.run(run(range(20)))))
```

```
import asyncio

async def first_good(coros, accept):
    """Return the first acceptable result and cancel the rest (hedged requests)."&quo
t;"
```

```
    tasks = [asyncio.create_task(c) for c in coros]
    try:
        for fut in asyncio.as_completed(tasks):
            res = await fut
            if accept(res):
                return res
    finally:
        for t in tasks:
            t.cancel()
    return None

async def slow(x, d):
    await asyncio.sleep(d)
    return x

print(asyncio.run(first_good([slow("a", .3), slow("b", .05)], lambda r: True)))
```

3.8 Testing, Mocking and Dependency Injection for LLM Code

Conceptual Deep Dive

Inject the model client as a parameter (or Protocol) so tests substitute a **fake** that returns canned outputs, records calls, and can simulate failures (timeouts, 429, malformed JSON). Layers: pure-function unit tests (parsers, chunkers, prompt builders), contract tests against recorded responses (VCR-style cassettes), and a small number of live smoke tests. Use pytest fixtures, pytest-asyncio, and property-based tests (hypothesis) for parsers. Edge cases: tests that pass only because the fake is too friendly; cassettes that leak API keys; time-dependent code (use an injected clock).

Cross-Connection Scenarios

Prompt/eval tests (10.4), CI (15.1), reliability fault injection (4.3).

Production Anti-Patterns

- Patching the SDK deep inside modules instead of injecting.
- Live-API tests in the default test run.
- Recording cassettes containing secrets.

Minimal Working Code

```
class FakeLLM:
    def __init__(self, outputs):
        self.outputs, self.calls = list(outputs), []

    def complete(self, prompt: str) -> str:
        self.calls.append(prompt)
        out = self.outputs.pop(0)
        if isinstance(out, Exception):
            raise out
        return out

def summarize(llm, text: str, retries: int = 2) -> str:
    for attempt in range(retries + 1):
        try:
            return llm.complete(f"Summarize: {text}")
        except TimeoutError:
            if attempt == retries:
                raise

def test_retries_then_succeeds():
    llm = FakeLLM([TimeoutError(), "short summary"])
    assert summarize(llm, "long text") == "short summary"
```

```
    assert len(llm.calls) == 2

test_retries_then_succeeds()
```

```
import re

def parse_score(text: str) -> int | None:
    m = re.search(r"\b([1-5])\b", text)
    return int(m.group(1)) if m else None

for s in ["Score: 4", "4/5", "no digits", "9 then 3"]:
    r = parse_score(s)
    assert r is None or 1 <= r <= 5
```

3.9 Data Engineering with Polars, DuckDB and Parquet

Conceptual Deep Dive

Most AI pipelines are data pipelines. **Parquet** is columnar, compressed and typed, so it reads only needed columns and is far smaller than CSV. **Polars** gives a lazy, multi-threaded DataFrame API that optimises the query plan (predicate and projection pushdown) and streams larger-than-memory data. **DuckDB** runs SQL directly over Parquet/CSV files with no server, ideal for ad-hoc analysis of logs and eval results. Patterns: partition datasets by date, store embeddings as list columns or separate files, keep raw immutable data separate from derived tables, and record schema versions. Edge cases: schema drift across files, nulls vs empty strings, time zones, very wide text columns blowing memory, and nondeterministic row order after parallel operations (sort explicitly before splitting).

Cross-Connection Scenarios

Eval result analysis (10), fine-tuning dataset preparation (12.2), log analysis (14.1), reproducible splits (15.4).

Production Anti-Patterns

- Loading a full multi-GB CSV into pandas to compute one aggregate.
- Writing intermediate results as pickles that break across versions.
- Random splits without a fixed seed or stable ordering.

Minimal Working Code

```
# pip install polars duckdb
import polars as pl

def eval_summary(path: str) -> pl.DataFrame:
    return (pl.scan_parquet(path)
            .filter(pl.col("split") == "test")
            .group_by("slice")
            .agg(pl.len().alias("n"), pl.col("correct").mean().alias("acc"
;),
                 pl.col("latency_ms").quantile(0.95).alias("p95_ms"))
            .sort("acc")
            .collect())

df = pl.DataFrame({"split": ["test"] * 4, "slice": ["a", "a&
quot;, "b", "b"],
                   "correct": [1, 0, 1, 1], "latency_ms": [100, 300, 120, 140]})
df.write_parquet("evals.parquet")
print(eval_summary("evals.parquet"))
```

```
import duckdb
```

```
def worst_slices(path: str, min_n: int = 2):
    return duckdb.sql(f"""
        SELECT slice, count(*) AS n, avg(correct) AS acc
        FROM read_parquet('{path}')
        GROUP BY slice HAVING count(*) >= {min_n}
        ORDER BY acc ASC LIMIT 5
    """).fetchall()

print(worst_slices("evals.parquet"))
```

3.10 Profiling and Performance Optimisation

Conceptual Deep Dive

Measure first. Tools: time.perf_counter for coarse timing, cProfile/snakeviz for CPU hot spots, py-spy for sampling a running process without code changes, line_profiler for hot lines, tracemalloc/memray for memory, and asyncio debug mode to find blocking calls (slow callback warnings). Typical wins in AI pipelines: batch API calls, reuse HTTP connections, cache embeddings by content hash, vectorise loops with NumPy, avoid repeated JSON (de)serialisation of large payloads, use orjson, and move CPU-heavy tokenisation or parsing to process pools. Edge cases: micro-benchmarks distorted by caching or warm-up, GPU timings requiring explicit synchronisation (torch.cuda.synchronize()), and optimisation of code that is 2% of wall time.

Cross-Connection Scenarios

Async patterns (3.1, 3.7), memory (3.4), embedding caching (8.6), benchmarking services (10.6).

Production Anti-Patterns

- Optimising before profiling.
- Timing GPU code without synchronisation.
- Premature complexity (custom C extensions) when batching would suffice.

Minimal Working Code

```
import time
from contextlib import contextmanager

@contextmanager
def timer(label: str, sink: dict):
    t0 = time.perf_counter()
    try:
        yield
    finally:
        sink[label] = sink.get(label, 0.0) + time.perf_counter() - t0

stats: dict[str, float] = {}
with timer("parse", stats):
    sum(i * i for i in range(200_000))
with timer("embed", stats):
    time.sleep(0.01)
print({k: round(v, 4) for k, v in stats.items()})
```

```
import asyncio
import time

async def find_blocking_calls():
    loop = asyncio.get_running_loop()
    loop.slow_callback_duration = 0.05         # log callbacks that block longer than 50 ms
    time.sleep(0.2)                            # deliberately blocking: shows up in debug logs

# run with:  asyncio.run(find_blocking_calls(), debug=True)
```